Apunte de repaso · Parcial 2 — Control de error

Codificación de canal · Teoría de la Información · Ejercicios de los parciales 2010–2025 resueltos paso a paso

Para qué sirve y cómo está armado

Es el equivalente del apunte maestro, pero **solo de control de error** y construido desde los ejercicios reales de los parciales, ordenados de lo más simple a lo más complejo. La idea es que aprendas **haciendo**: cada ejercicio tiene una receta, el desarrollo completo y una verificación.

0. Dónde estás, plan de repaso y mapa de qué cae
1. Caja de herramientas (lo mínimo de memoria)
2. ISBN y base prima
3. Hamming y arreglos geométricos
4. Códigos cíclicos: el corazón del parcial
5. Entrelazado y ráfagas
6. Reed–Solomon (la tabla)
7. Teóricas con respuesta
8. ⚠ Errores en resoluciones de alumnos
9. Ejercicios propuestos + soluciones
10. Chuleta de una página

Verificación: todos los números (polinomios generadores, divisiones, pesos de *todas* las palabras, matrices con $G \cdot H^T = 0$, síndromes, ISBN) fueron recalculados con un programa, no a mano. Donde una resolución de alumno no coincide con el cálculo está marcado con ⚠ y explicado en la sección 8.

Fuentes: parciales 2 de 2010, 2012 (T1, T2), 2013 (T1, T2), 2020 (resolución de J. Arce), 2025 (resolución de B. Melani), «Preguntas teóricas parcial 2», y las diapositivas del Área Temática 4 (Control de error 2026).

0. Dónde estás parado, plan de repaso y mapa de qué cae

0.1 Dónde estás

- **Ya visto en clase** (diapositivas del Área 4 hasta la 9, ISBN): idea de redundancia, distancia de Hamming, $d_{min} \geq 2t+1$, detector = $d_{min}-1$, bit de paridad, Hamming (7,4), Hamming + bit de paridad, arreglos gráficos, ISBN de 10 y 13 dígitos y la base prima.
- **Todavía sin práctica** (y es lo que más puntos mueve): matrices **G** y **H**, códigos **cíclicos** (obtener g(x) desde h(x), k, d_{min}, chequeadores como combinación de bits de información), **decodificar con el resto/síndrome**, circuito de h(x), **entrelazado** y la **tabla Reed–Solomon**.

Nota: la diapositiva 9 con ISBN es del PDF del **Área 4** (Control de error). En el PDF del Área 1 esa misma página es «Canales de comunicación y capacidad».

0.2 Plan de repaso (en este orden)

Etapa	Tema	Estás listo cuando…	Hacer
E0	Notación y capacidad	pasás cadena de bits ↔ polinomio sin dudar y sabés de memoria la tabla d_{min} → corrige / detecta	§1 y P0
E1	ISBN-10, ISBN-13, base prima	sacás el dígito faltante en 2 minutos y explicás por qué la base es prima	§2 · P1, P2
E2	Hamming (7,4) y Hamming + BP	codificás, calculás el síndrome y corregís; explicás los 4 casos (S, BP)	§3.1 · P3
E3	Arreglos geométricos	calculás $d_{min} = d_1 \cdot d_2$ y localizás errores por fila y columna	§3.2
E4	Cíclicos I: k, g(x), d_{min}	dado h(x) y n obtenés k, g(x) (división) y d_{min} (enumerando palabras)	§4.1, 4.2 · P4, P5
E5	Cíclicos II: G, H, chequeadores	armás G (sistemática y por desplazamientos), H, las ecuaciones de cada chequeador y verificás $G \cdot H^T = 0$	§4.2 (pasos 5–6), 4.3
E6	Cíclicos III: decodificar	dividís R(x) por g(x), leés el resto como síndrome, corregís y explicás qué pasa con un error triple	§4.2 (7–8), 4.3, 4.4 · P4
E7	Circuito y entrelazado	dibujás el registro de h(x) y calculás la ráfaga t·L	§4.2 (10), §5
E8	Reed–Solomon y teóricas	completás la tabla RS y respondés las teóricas fijas sin mirar	§6, §7 · P6
E9	Simulacro	resolvés el control de error de 2020 y de 2025 completos, sin apuntes, en tiempo de examen	§9 · P8

Orden sugerido: E0–E3 refuerzan lo que ya viste; E4–E6 son lo nuevo y es donde conviene invertir más tiempo (cae en *todos* los parciales); E7–E8 son rápidos una vez que dominás los cíclicos. Hacé primero el ejercicio sin mirar la solución y comparás.

0.3 Mapa de qué cae (por año)

Tipo de ejercicio	2010	2012 T1	2012 T2	2013 T1	2013 T2	2020	2025
Capacidad: d_{min}, t, detección	●	●	●	●	●	●	●
$g(x) = (x^n+1)/h(x)$	·	●	●	●	·	●	dado
Chequeadores = combinación de bits de información	●	●	●	●	●	·	·
Matrices G y/o H	·	·	·	·	● (G)	●	●
Decodificar R(x), detectar y corregir	●	●	●	●	·	·	●
Síndrome, error triple	·	·	·	·	·	●	·
Circuito de h(x)	●	·	·	·	·	·	·
Entrelazado / ráfagas	● (teór.)	●	●	●	●	·	●
ISBN (dígito faltante / validar)	·	·	● (teór.)	·	●	●	● (13)
Base prima	·	·	●	·	·	●	·
Arreglos geométricos	·	● (teór.)	·	·	·	·	●
Tabla Reed–Solomon	·	·	● (teór.)	·	·	·	·
Propiedades de g(x)	·	● (teór.)	·	·	·	·	·

Lo que se repite

Tres códigos explican casi todos los exámenes: **(12,3)** con $h = x^3+x^2+x+1$ (2010, 2013 T2, 2020), **(14,4)** con $h = x^4+x^2+x+1$ (2012 T2, 2013 T1, 2025) y **(15,4)** con $h = x^4+x+1$ (2012 T1). Los resolvemos completos en la sección 4: si los dominás, los dominás a todos, y si cambia el polinomio el procedimiento es idéntico.

1. Caja de herramientas

1.1 Notación: «mayores potencias a la derecha»

En los parciales, una cadena de bits `b₀ b₁ b₂ … bₙ₋₁` es el polinomio $\mathbf{b_0 + b_1x + b_2x^2 + \ldots + b_{n-1}x^{n-1}}$: el **primer bit es $\mathbf{x^0}$**.

Ejemplo (2025): `11010001101` → posiciones con 1: 0, 1, 3, 7, 8, 10 → $\mathbf{1 + x + x^3 + x^7 + x^8 + x^{10}}$.

> **Regla para no perder puntos**
>
> Mantené una sola convención. Para **dividir a mano** conviene escribir el polinomio con la **mayor potencia a la izquierda** (invertí la cadena del enunciado); al final volvé a la convención del enunciado para escribir resultados. En este apunte las tablas de división están así y se aclara en cada una.

1.2 Aritmética módulo 2

Suma = XOR (0+0=0, 0+1=1, **1+1=0**); producto = AND. Restar es lo mismo que sumar. Sumar dos polinomios = XOR bit a bit de sus cadenas. En una división polinómica, cada paso es un XOR del divisor desplazado.

1.3 Parámetros del código

- **n**: longitud de la palabra · **k**: bits de información · **n−k**: chequeadores (bits de control). Hay 2^k palabras válidas entre 2^n posibles.
- **Redundancia** (como la usa la cátedra) = n/k; cuanto más cerca de 1, mejor. Tasa = k/n.
- Código **lineal**: suma de dos palabras válidas es válida. Entonces $\mathbf{d_{min}}$ **= peso mínimo de una palabra válida no nula** (la distancia entre dos palabras es el peso de su suma, que también es válida).

1.4 Capacidad del código

Con distancia mínima d_{min}: **corrige t** errores con $\mathbf{d_{min} \geq 2t+1}$, es decir $\mathbf{t = \lfloor (d_{min}-1)/2 \rfloor}$; como **detector** puro detecta $\mathbf{d_{min}-1}$. Si se usa a la vez para corregir t y detectar hasta s ≥ t: $\mathbf{d_{min} \geq t+s+1}$.

d_{min}	Solo detector ($d_{min}-1$)	Solo corrector (t)	Corrector + detector
2	1	0	—
3	2	1	corrige 1
4	3	1	corrige 1 y detecta 2
5	4	2	corrige 2
6	5	2	corrige 2 y detecta 3
7	6	3	corrige 3
8	7	3	corrige 3 y detecta 4
9	8	4	corrige 4
10	9	4	corrige 4 y detecta 5

Coincide con la tabla de la cátedra (p. ej. d_{min} = 6 «corrige doble y detecta triple»; d_{min} = 10 «corrige 4, detecta 5 / como detector detecta 9»).

1.5 Códigos cíclicos: qué es cada polinomio

Símbolo	Qué es	Cómo se obtiene
h(x)	polinomio de **chequeo**, grado **k**	lo da el enunciado (los parciales viejos lo llaman «generador», pero luego piden g(x))
g(x)	polinomio **generador**, grado **n−k**	$g(x) = (x^n + 1) / h(x)$ (división exacta)
V(x)	palabra válida (múltiplo de g)	no sistemática: $V = i(x)\cdot g(x)$ · sistemática: $V = x^{n-k}\cdot i(x) + r(x)$, con $r = x^{n-k}\cdot i(x) \bmod g$

Símbolo	Qué es	Cómo se obtiene
R(x)	palabra recibida	R = V + e (e = patrón de errores)
síndrome	resto de R(x) ÷ g(x)	0 ⇒ válida · ≠ 0 ⇒ hubo error; el resto identifica el patrón

Propiedades de g(x) (teórica fija del parcial)

1. g(x) es de grado **n−k**.
2. g(x) es **factor de x^n+1**.
3. Tiene coeficiente 1 en el término independiente y en el de mayor grado.
4. Es el polinomio de **menor grado** del código (y primo, según la cátedra).
5. Toda palabra válida V(x) es **múltiplo de g(x)**, y todo múltiplo de g(x) de grado menor que n es palabra válida.
6. Todo **desplazamiento cíclico** de una palabra válida es otra palabra válida.

Atajo de la cátedra: como g(x) es una palabra del código, $d_{min} \leq$ *cantidad de términos de g(x)*. En los tres códigos de los parciales da igualdad (6, 6 y 8 términos), pero el atajo no es un teorema: la certeza sale de enumerar las palabras (lo hacemos abajo).

Atajo de validez: un polinomio **no nulo de grado menor que n−k** nunca es palabra válida (no puede ser múltiplo de g).

2. ISBN y base prima (Etapa 1)

2.1 Receta ISBN-10 con un dígito desconocido

Receta (código «pesado», base 11)

1. Escribí los 10 dígitos y marcá el desconocido **?**. El símbolo de control (posición 10) puede ser **X** = 10.
2. Pesos: **10, 9, 8, …, 1** (la posición 1 pesa 10; la 10 pesa 1).
3. Calculá dígito × peso y sumá los conocidos: **S**.
4. Con el peso **w** de la posición del ?: resolvé **S + w·x ≡ 0 (mod 11)**, o sea $x \equiv -S \cdot w^{-1} \pmod{11}$.
5. Verificá: la suma total debe ser múltiplo de 11. Si da 10 solo vale como X en el verificador.

Inversos módulo 11 (para no calcular cada vez): 1→1, 2→6, 3→4, 4→3, 5→9, 6→2, 7→8, 8→7, 9→5, 10→10.

2.2 Ejercicios resueltos

2013 T2 ISBN `1-122-?3445-5`: encontrar el símbolo ? y la capacidad del código

Posición	1	2	3	4	5	6	7	8	9	10
Dígito	1	1	2	2	?	3	4	4	5	5
Peso	10	9	8	7	6	5	4	3	2	1
Dígito × peso	10	9	16	14	?	15	16	12	10	5

S = 10+9+16+14+15+16+12+10+5 = **107**; 107 = 9·11 + 8, así que S ≡ 8. El ? está en la posición 5, peso **6**.

$6x \equiv -8 \equiv 3 \pmod{11} \rightarrow x \equiv 3 \cdot 6^{-1} = 3 \cdot 2 = 6$.

? = 6 → ISBN 1-122-63445-5. Verificación: 107 + 6·6 = 143 = 13·11 ✓

2020 ISBN `3-540-?3218-9` (versión de 10 dígitos)

Posición	1	2	3	4	5	6	7	8	9	10
Dígito	3	5	4	0	?	3	2	1	8	9
Peso	10	9	8	7	6	5	4	3	2	1
Dígito × peso	30	45	32	0	?	15	8	3	16	9

S = 30+45+32+0+15+8+3+16+9 = **158**; 158 = 14·11 + 4, S ≡ 4. Peso del ?: 6. 6x ≡ −4 ≡ 7 → x ≡ 7·2 = 14 ≡ **3**.

? = 3 → 3-540-33218-9. Verificación: 158 + 18 = 176 = 16·11 ✓ (es justo la suma 176 que aparece en la diapositiva 9 para este ISBN)

⚠ La resolución de alumna del archivo llega a «6» por un error de suma (usa 113 en lugar de 158). Ver sección 8.

2012 T2 ISBN `234323?323`

Posición	1	2	3	4	5	6	7	8	9	10
Dígito	2	3	4	3	2	3	?	3	2	3
Peso	10	9	8	7	6	5	4	3	2	1
Dígito × peso	20	27	32	21	12	15	?	9	4	3

S = 20+27+32+21+12+15+9+4+3 = **143** = 13·11 → S ≡ 0. Peso del ?: 4 → 4x ≡ 0 (mod 11) → como 4 es invertible, x ≡ 0.

? = 0. Verificación: 143 + 0 = 143 = 13·11 ✓

2.3 Capacidad del código ISBN y base prima (teóricas fijas)

Capacidad del ISBN

- **$d_{min} = 2$**: detecta **un** error (si cambia un solo dígito, la suma cambia en $w \cdot \delta \neq 0 \bmod 11$ y deja de ser múltiplo de 11). Como los pesos son todos distintos, también detecta el **intercambio de dos dígitos adyacentes**.
- **No corrige**: sabe que algo falló pero no dónde. Solo se corrige con una hipótesis (p. ej. «el erróneo es el verificador» o «sé cuál posición está mal», como en los ejercicios con ?).
- Dos errores que se compensan pasan inadvertidos: el código resultante es válido pero corresponde a **otro libro** (error indetectable).

¿Por qué base prima? (2012 T2 y 2020)

Para decodificar hay que resolver **$w \cdot x \equiv c \pmod b$**. Si la base **b es prima**, cualquier peso $w \neq 0$ tiene inverso y la solución es **única**: no hay ambigüedad al decodificar (la cátedra: «la base prima garantiza unicidad en el proceso de decodificación»).

Contraejemplo con base 10: $2x \equiv 4 \pmod{10}$ tiene dos soluciones, $x = 2$ y $x = 7$. Con base 11, $2x \equiv 4 \rightarrow x = 2$ y nada más.

Base 37 (diapositiva 9)

Para códigos alfanuméricos (0–9, A–Z, espacio = 37 símbolos) se usa base 37 **sin pesos por posición**: se suman los valores de los caracteres y el verificador completa a múltiplo de 37. Ejemplo de la diapositiva: AZ9-? → 10 + 35 + 9 = 54; (54 + ?) mod 37 = 0 → ? = 20 = **K**.

2.4 ISBN-13 (2025)

2025 **ISBN-13 `978-1-80056-662-3`: ¿es válido? Si no, ¿se puede corregir?**

Receta ISBN-13 (base 10)

Pesos **1, 3, 1, 3, …** (el último, posición 13, pesa 1). Válido ⇔ suma de dígito × peso ≡ 0 (mod 10). Dígito verificador = (10 − (suma de los 12 primeros mod 10)) mod 10.

Posición	1	2	3	4	5	6	7	8	9	10	11	12	13
Dígito	9	7	8	1	8	0	0	5	6	6	6	2	3
Peso	1	3	1	3	1	3	1	3	1	3	1	3	1
Dígito × peso	9	21	8	3	8	0	0	15	6	18	6	6	3

Suma total = **103** → 103 mod 10 = 3 ≠ 0 ⇒ **el código NO es válido**.

¿Corregir? Solo con una hipótesis. Si el erróneo es el verificador: la suma de los 12 primeros es 100 → verificador = (10 − 0) mod 10 = **0** → `978-1-80056-662-0` es válido (verificación: 100 + 0 = 100 ≡ 0 ✓).

Sin esa hipótesis solo se **detecta**: cada una de las 13 posiciones tiene exactamente un dígito que haría válido el código (en una posición de peso 1 hay que sumar 7 mod 10; en una de peso 3, restar 1 mod 10), así que hay 13 correcciones posibles y no se sabe cuál es la verdadera (con 978 fijo siguen quedando 10).

ISBN-13 usa módulo 10 (no primo) pero con pesos 1 y 3, que son coprimos con 10 y por eso invertibles. Detecta cualquier error simple; no detecta el intercambio de dos dígitos adyacentes que difieren en 5.

3. Hamming y arreglos geométricos (Etapas 2 y 3)

3.1 Hamming (7,4)

Posiciones 1 a 7: `C1 C2 I1 C3 I2 I3 I4`. Los chequeadores ocupan las posiciones potencia de 2 (1, 2, 4). Cada síndrome mira las posiciones cuya numeración binaria tiene un 1 en ese bit:

- $S_1 = C_1 \oplus I_1 \oplus I_2 \oplus I_4$ (posiciones 1, 3, 5, 7)
- $S_2 = C_2 \oplus I_1 \oplus I_3 \oplus I_4$ (posiciones 2, 3, 6, 7)
- $S_3 = C_3 \oplus I_2 \oplus I_3 \oplus I_4$ (posiciones 4, 5, 6, 7)

$(S_3\ S_2\ S_1)$ en binario = posición del error; si es 000 no hay error. $d_{min} = 3 \rightarrow$ corrige 1, detecta 2. Es un código *perfecto*: $2^3 = 8 = 1 + 7$ (cada síndrome sirve a una posición o a «sin error»).

> **Ejemplo completo**
>
> **Codificar** $I_1I_2I_3I_4 = 1\ 0\ 1\ 1$: $C_1 = I_1 \oplus I_2 \oplus I_4 = 1 \oplus 0 \oplus 1 = 0$; $C_2 = I_1 \oplus I_3 \oplus I_4 = 1 \oplus 1 \oplus 1 = 1$; $C_3 = I_2 \oplus I_3 \oplus I_4 = 0 \oplus 1 \oplus 1 = 0$. Palabra: `0 1 1 0 0 1 1`.
>
> **Canal** cambia la posición 6: recibido `0 1 1 0 0 0 1`.
>
> **Decodificar**: $S_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$; $S_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$; $S_3 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$. $(S_3S_2S_1) = 110 = 6 \rightarrow$ se invierte la posición 6 → `0110011` ✓.

Hamming + bit de paridad ($d_{min} = 4$)

Se agrega BP = paridad de los 7 bits (código (8,4)). Con el síndrome de Hamming S y el control de BP se distinguen 1 y 2 errores:

Síndrome S	BP (¿falla la paridad total?)	Conclusión
S = 0	no falla	sin error (o error múltiple muy improbable)
S = 0	falla	error simple en el propio BP
S ≠ 0	falla (número impar de errores)	error simple en la posición S → **se corrige**
S ≠ 0	no falla (número par de errores)	**doble error**: se detecta, no se corrige → pedir retransmisión

Resultado: $d_{min} = 4 \rightarrow$ **corrector simple y detector doble** (idéntico a la fila $d_{min} = 4$ de la tabla 1.4).

3.2 Arreglos geométricos (rectangular)

Regla: en un código producto (arreglo rectangular) con código por filas de distancia $\mathbf{d_1}$ y código por columnas de distancia $\mathbf{d_2}$, la distancia mínima es $\mathbf{d_{min} = d_1 \cdot d_2}$.

2025 / 2012 T1 Arreglo de palabras Hamming con paridad por filas y por columnas

El arreglo tiene 3 filas; cada fila es una palabra Hamming de un bit de información (C_1 C_2 I) seguida de su bit de paridad de fila BP; debajo hay un bit de paridad por cada columna.

C_{10}	C_{20}	I_{10}	BPP_0
C_{11}	C_{21}	I_{11}	BPP_1
C_{12}	C_{22}	I_{12}	BPP_2
BPP_3	BPP_4	BPP_5	

Paso 1 – código de cada fila. Hamming (3,1) tiene d = 3: sus únicas palabras son 000 y 111 ($C_1 = C_2 = I$). Al agregar la paridad de fila BP queda el (4,1) con $\mathbf{d_1 = 4}$ (0000 o 1111).

Paso 2 – código de cada columna. Paridad simple: $\mathbf{d_2 = 2}$.

Paso 3 – producto. $d_{min} = d_1 \cdot d_2 = 4 \cdot 2 = \mathbf{8}$ (con el bit de la esquina, la «paridad de las paridades»). Sin la esquina, como en el dibujo, queda 7. Lo comprobé enumerando las 8 palabras:

Información (I_{10} I_{11} I_{12})	Peso sin esquina	Peso con esquina
0 0 1	7	8
0 1 0	7	8
0 1 1	8	8
1 0 0	7	8
1 0 1	8	8
1 1 0	8	8
1 1 1	15	16

d_{min} = 8 (7 si no se agrega la esquina) → corrige t = 3, detecta 6 o 7.

⚠ La resolución de alumna usa d_1 = 3 (Hamming sin el bit de paridad de fila) y obtiene d_{min} = 6, t = 2. Es una cota inferior válida (el código real tiene $d_{min} \geq 7$), pero no es ajustada. El enunciado dice «cada fila es una palabra Hamming y existen paridades por filas y por columnas»; como depende de cómo se lea el BP de fila, conviene confirmar con la cátedra qué valor esperan. Ver sección 8.

Cómo se reconocen y corrigen los errores:

1. **Por fila**: síndrome Hamming (S_2S_1 = posición) + BP de fila (tabla de 3.1): corrige un error por fila y distingue 1 de 2 errores.
2. **Por columna**: la paridad de cada columna falla si tiene un número impar de errores.
3. **Cruce**: si falla una fila y una columna, el error está en su intersección; sirve cuando la fila no puede corregir sola (2 errores en la fila).

2025 ¿Es válida esta palabra recibida?

1	0	0	1
0	0	0	1
1	1	1	1
0	0	0	

- **Fila 1** (1 0 0 | 1): $S_1 = C_1 \oplus I = 1$, $S_2 = C_2 \oplus I = 0$ → Hamming «apunta» a la posición 1, pero la paridad de fila **sí** da par (1+0+0+1 = 0): contradicción → hay 2 errores en la fila.
- **Fila 2** (0 0 0 | 1): S = 00 pero falla la paridad → el error está en el BP de la fila 2.
- **Fila 3** (1 1 1 | 1): válida.
- **Columnas**: col 1 = (1,0,1) con BP 0 → ok · col 2 = (0,0,1) con BP 0 → **falla** · col 3 = (0,0,1) con BP 0 → **falla**.

No es válida (fallan filas 1 y 2 y las columnas 2 y 3). La alumna menciona solo la fila 2 y la columna 2; la columna 3 también falla.

Corrección por mínima distancia: las 8 palabras válidas están a distancias 3, 5, 5, 6… de lo recibido; la única a distancia ≤ t = 3 es la que corresponde a información 1 0 1 (filas 1111 / 0000 / 1111, paridades de columna 000). Distancia 3 ≤ t → corrección única. A mano se llega con el procedimiento fila–columna.

4. Códigos cíclicos: el corazón del parcial (Etapas 4–7)

Receta general (aplica a TODOS los parciales)

1. **k** = grado de h(x); chequeadores = n − k.
2. **g(x) = (x^n+1) / h(x)** por división polinómica (resto 0).
3. **d_{min}**: listá las 2^k−1 palabras no nulas V = m(x)·g(x) y tomá el **peso mínimo**. Atajo: d_{min} ≤ #términos de g.
4. **Capacidad**: t = ⌊(d_{min}−1)/2⌋; detecta d_{min}−1.
5. **Matriz generadora**: sistemática G = [P | I_k] con la fila i de P = (x^{n-k+i} mod g). O sin sistematizar: filas = g(x), x·g(x), x^2·g(x), …
6. **Matriz chequeadora**: H = [I_{n-k} | P^T]. Comprobá **$G \cdot H^T = 0$**.
7. **Chequeadores**: la columna j de P dice qué bits de información suma el chequeador c_j.
8. **Decodificar**: r(x) = R(x) mod g(x). r = 0 ⇒ válida. Si no, buscá el patrón de error e de peso ≤ t con e mod g = r y corregí V = R ⊕ e.
9. **Circuito**: registro de k celdas realimentado según h(x).
10. **Entrelazado**: con L palabras entrelazadas se corrigen ráfagas de t·L bits.

4.1 Calentamiento: el cíclico (7,4) con g(x) = x^3 + x + 1

Es el ejemplo de la cátedra; es chico y sirve para ver todos los pasos con cuentas cortas antes de ir a los códigos de los parciales.

(a) n = 7 y deg g = 3 ⇒ **k = 4**. Verificación de que g divide a x^7+1 (cociente = h):

Paso	Resto parcial (mayor potencia a la izquierda)	Término del cociente	Divisor desplazado	Resto nuevo (suma mod 2)
1	`10000001`	x^4	`10110000`	`00110001`
2	`00110001`	x^2	`00101100`	`00011101`
3	`00011101`	x	`00010110`	`00001011`
4	`00001011`	1	`00001011`	`00000000`

Resto 0 ✓; el cociente es h(x) = x^4 + x^2 + x + 1.

(b) d_{min}: las 15 palabras no nulas pesan 3 (×7), 4 (×7) y 7 (×1) → **d_{min} = 3, t = 1** (es el Hamming (7,4)).

(c) Codificar la información i(x) = 1 + x^2 (bits `1010`):

- No sistemática: V = i·g = (1+x^2)(1+x+x^3) = **x^5 + x^2 + x + 1** → `1110010`.
- Sistemática: x^3·i(x) = x^3 + x^5; dividiendo por g el resto es **x^2**; V = x^5 + x^3 + x^2 → `0011010` (chequeadores `001` en las posiciones 0–2, información `1010` en las 3–6).

(d) Síndromes de error simple (resto de x^i ÷ g): los 7 son distintos y no nulos, por eso corrige 1 error.

Error en	x^i mod g	Síndrome (bits)
1	1	100
x	x	010
x^2	x^2	001
x^3	x + 1	110
x^4	x^2 + x	011
x^5	x^2 + x + 1	111
x^6	x^2 + 1	101

(e) Decodificar R = `0011110` (la palabra de (c) con un error). R(x) = x^5 + x^4 + x^3 + x^2 → resto de R ÷ g = **x^2 + x** ≠ 0 → no es válida. Ese resto es el de x^4 en la tabla → error en la posición 4 → se invierte el bit 4: V = `0011010`; información = `1010` ✓.

4.2 Código (12, 3) con $h(x) = x^3 + x^2 + x + 1$ 2010 2013 T2 2020

Enunciado tipo (2020): «Sea $h(x) = x^3+x^2+x+1$ de un código (12, k, d_{min}). A) k. B) d_{min} indicando cualidades como corrector y como detector. C) polinomio generador g(x). D) matriz generadora G y chequeadora H. E) qué sucede en la decodificación ante un **error triple**; y si se comparara el síndrome con las filas de H^T.»
2010: capacidad, chequeadores como combinación de bits de información, decodificar 110011001110 con corrección, dibujar el circuito de h(x). **2013 T2:** capacidad, G y chequeadores, entrelazar 20 códigos.

Paso 1 · k

El grado de h(x) es 3 → **k = 3** bits de información; n − k = 12 − 3 = **9** chequeadores. Hay $2^3 = 8$ palabras válidas.

Paso 2 · polinomio generador $g(x) = (x^{12} + 1) / h(x)$

Cada fila resta (suma mod 2) el divisor desplazado. Los términos del cociente son el polinomio buscado.

Paso	Resto parcial (mayor potencia a la izquierda)	Término del cociente	Divisor desplazado	Resto nuevo (suma mod 2)
1	`1000000000001`	x^9	`1111000000000`	`0111000000001`
2	`0111000000001`	x^8	`0111100000000`	`0000100000001`
3	`0000100000001`	x^5	`0000111100000`	`0000011100001`
4	`0000011100001`	x^4	`0000011110000`	`0000000010001`
5	`0000000010001`	x	`0000000011110`	`0000000001111`
6	`0000000001111`	1	`0000000001111`	`0000000000000`

$g(x) = x^9 + x^8 + x^5 + x^4 + x + 1$ → en bits (mayores potencias a la derecha): `1100110011` (grado 9 = n−k ✓, 6 términos)

Paso 3 · d_{min} enumerando las palabras

Palabra = m(x)·g(x) (no sistemática). Se listan las 7 no nulas:

Info m (bits)	m(x)	V = m·g (bits)	Peso
100	1	1100 1100 1100	6
010	x	0110 0110 0110	6
110	1 + x	1010 1010 1010	6
001	x^2	0011 0011 0011	6
101	$1 + x^2$	1111 1111 1111	12
011	$x + x^2$	0101 0101 0101	6
111	$1 + x + x^2$	1001 1001 1001	6

$d_{min} = 6$ → $t = \lfloor 5/2 \rfloor = 2$ | detecta $d_{min}-1 = 5$ | corrige 2 y detecta 3 a la vez ($6 \geq 2+3+1$) | redundancia n/k = 4

Coincide con el atajo: g tiene 6 términos. Control con la cota de Hamming: $1 + 12 + 66 = 79 \leq 2^9 = 512$ ✓ (no es un código perfecto: sobran síndromes).

Paso 4 · G y H en forma sistemática y chequeadores

Con «mayores potencias a la derecha» los chequeadores quedan en las posiciones 0–8 y la información en las 9–11 (x^9, x^{10}, x^{11}). La fila i de P es **x^{9+i} mod g**:

Bit de información	x^{9+i} mod g	Fila de P (9 bits)
x^9	$x^8 + x^5 + x^4 + x + 1$	110011001
x^{10}	$x^8 + x^6 + x^4 + x^2 + 1$	101010101
x^{11}	$x^8 + x^7 + x^4 + x^3 + 1$	100110011

G (3 × 12) = [P | I_3]

```
1 1 0 0 1 1 0 0 1 | 1 0 0
1 0 1 0 1 0 1 0 1 | 0 1 0
1 0 0 1 1 0 0 1 1 | 0 0 1
```

H (9 × 12) = [I_9 | P^T]

```
1 0 0 0 0 0 0 0 0 | 1 1 1
0 1 0 0 0 0 0 0 0 | 1 0 0
0 0 1 0 0 0 0 0 0 | 0 1 0
0 0 0 1 0 0 0 0 0 | 0 0 1
0 0 0 0 1 0 0 0 0 | 1 1 1
0 0 0 0 0 1 0 0 0 | 1 0 0
0 0 0 0 0 0 1 0 0 | 0 1 0
0 0 0 0 0 0 0 1 0 | 0 0 1
0 0 0 0 0 0 0 0 1 | 1 1 1
```

Verificación: cada fila de G por cada fila de H da 0 mod 2 ($G \cdot H^T = 0$) ✓.

Cada chequeador como combinación de los bits de información (i_0, i_1, i_2 = bits en las posiciones 9, 10, 11):

c_0	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8
$i_0 \oplus i_1 \oplus i_2$	i_0	i_1	i_2	$i_0 \oplus i_1 \oplus i_2$	i_0	i_1	i_2	$i_0 \oplus i_1 \oplus i_2$

Es decir: $c_0=c_4=c_8 = i_0 \oplus i_1 \oplus i_2$; $c_1=c_5 = i_0$; $c_2=c_6 = i_1$; $c_3=c_7 = i_2$. (Con G sin sistematizar —filas g, x·g, x^2·g— el código es el mismo conjunto de 8 palabras; cambia qué mensaje corresponde a cada una.)

> **G sin sistematizar (como en la resolución 2025)**
>
> Filas = g(x), x·g(x), x^2·g(x)
>
> ```
> 1 1 0 0 1 1 0 0 1 1 0 0
> 0 1 1 0 0 1 1 0 0 1 1 0
> 0 0 1 1 0 0 1 1 0 0 1 1
> ```
>
> Funciona igual para generar el código; la desventaja es que la información queda mezclada con los chequeadores y no se lee directo.

Paso 5 · Decodificar el flujo recibido (2010)

Recibido: `110011001110` → $R(x) = x^{10} + x^9 + x^8 + x^5 + x^4 + x + 1$.

División de R(x) por g(x) (mayor potencia a la izquierda: se escribe invertido el flujo del enunciado):

Paso	Resto parcial (mayor potencia a la izquierda)	Término del cociente	Divisor desplazado	Resto nuevo (suma mod 2)
1	`11100110011`	x	`11001100110`	`00101010101`

El resto es $\mathbf{x^8 + x^6 + x^4 + x^2 + 1}$ (bits `101010101`) ≠ 0 → **la palabra no es válida**.

Localizar el error: se busca qué x^i deja ese mismo resto. Para i < 9 el resto sería x^i (un solo 1); acá hay varios 1, así que el error está en la parte de información, y el resto coincide con la fila de P de x^{10}: $x^{10} \bmod g = x^8 + x^6 + x^4 + x^2 + 1$ ✓. (Equivalente por matriz: el síndrome $S = R \cdot H^T$ es la columna 10 de H.)

Corrección: se invierte el bit 10: `110011001110` → `110011001100`. Comprobación: es la primera fila de G → válida; su información (posiciones 9–11) es `100`.

Error simple en x^{10} → palabra válida 110011001100, información 1 0 0

Paso 6 · Errores dobles y triples (2020, inciso E)

Cómo se decodifica en general: se divide R(x) por g(x); el resto es el **síndrome**. Se lo compara con las **filas de H^T** (= columnas de H = síndromes de errores simples): una coincidencia → error simple en esa posición; la suma (XOR) de dos filas → error doble en esas dos posiciones; etc.

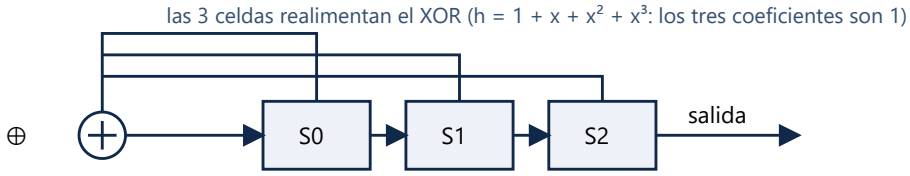
Error triple: como t = 2, el decodificador solo busca patrones de peso ≤ 2. Con 3 errores el síndrome es el XOR de 3 filas de H^T y **no coincide con ninguna fila ni con ningún XOR de dos filas**: el error se **detecta** (síndrome ≠ 0) pero **no se corrige** (hay que pedir retransmisión). Esto nunca se confunde con un error corregible: si un patrón de peso ≤ 2 diera el mismo síndrome, la diferencia entre ambos patrones sería una palabra válida de peso ≤ 5 < 6 = d_{min}, que no existe.

Ejemplo: palabra válida `111111111111` con errores en las posiciones 0, 1 y 2 → resto `111000000` (es el XOR de las filas de x^0, x^1 y x^2). Como no coincide con ninguna fila de H^T ni con la suma de dos de ellas, se detecta sin corregir.

Si en cambio hubiera **2 errores**, el síndrome es el XOR de dos filas de H^T (y es único: lo verifiqué para todos los patrones de peso ≤ 2) → se corrigen.

Paso 7 · Circuito de h(x) (2010, inciso D)

Un código cíclico con polinomio de chequeo h(x) de grado k se implementa con un **registro de desplazamiento de k = 3 celdas** cuyas realimentaciones (compuertas XOR) corresponden a los coeficientes de h(x) que valen 1: acá $h = 1 + x + x^2 + x^3$ → **las tres celdas** entran al XOR, y la suma se reinyecta a la entrada. En cada pulso el registro se corre un lugar y el bit que sale es el siguiente bit de la palabra.



Funcionamiento (comprobado): cargando la «semilla» de información `1 1 0` (S_0, S_1, S_2 = c_0 c_1 c_2), la secuencia de salida es $c_{i+3} = c_i \oplus c_{i+1} \oplus c_{i+2}$ → `110011001100` = una palabra válida (la fila g del código). Cargar 1 0 0 da 100110011001, 1 0 1 da 101010101010, etc. Estas 8 secuencias son justamente las 8 palabras válidas del código (g(x)·m(x)).

Lo que se evalúa en el parcial es que las conexiones coincidan con los coeficientes de h(x). Hay dos arquitecturas (Fibonacci y Galois, esta última es la usada en control de error); como h(x) de este código es palíndromo, el dibujo no cambia entre ambas. Confirmá con la cátedra el estilo de dibujo que esperan.

Paso 8 · Entrelazado (2010 y 2013 T2)

Con t = 2, entrelazando **L** palabras se corrigen ráfagas de **t·L** bits: **L = 7 → 14 bits** (2010); **L = 20 → 40 bits** (2013 T2). Más detalle en la sección 5.

4.3 Código (14, 4) con $h(x) = x^4 + x^2 + x + 1$ 2012 T2 2013 T1 2025

Enunciado tipo (2012 T2 / 2013 T1): «(14, K?, d_{min}?) con $h(X) = X^4+X^2+X+1$. a) g(X). b) capacidades como corrector y/o detector, encontrando d_{min}. c) cada chequeador como combinación lineal de qué dígitos de información. d) ¿$R(x) = X^8+X^4$ es válido? Si no, corregirlo. e) capacidad de corrección en ráfaga si se entrelazan 20 códigos.»
2025: dan directamente **G(x) = 11010001101** (mayores potencias a la derecha) y piden capacidad, matrices G y H, validar R(x) = 11110001101000, corregir, y entrelazar 25.

Paso 1 · k y g(x)

Si te dan h: deg h = 4 → **k = 4**, n−k = 10. Si te dan g (2025): deg g = 10 → k = 14 − 10 = 4 y $h = (x^{14}+1)/g = 1 + x + x^2 + x^4$ (bits `11101`).

Paso	Resto parcial (mayor potencia a la izquierda)	Término del cociente	Divisor desplazado	Resto nuevo (suma mod 2)
1	`100000000000001`	x^{10}	`101110000000000`	`001110000000001`
2	`001110000000001`	x^8	`001011100000000`	`000101100000001`
3	`000101100000001`	x^7	`000101110000000`	`000000010000001`
4	`000000010000001`	x^3	`000000010111000`	`000000000111001`
5	`000000000111001`	x	`000000000101110`	`000000000010111`
6	`000000000010111`	1	`000000000010111`	`000000000000000`

g(x) = $x^{10} + x^8 + x^7 + x^3 + x + 1$ → `11010001101` (6 términos; es exactamente el G(x) del enunciado 2025)

Paso 2 · d_{min} enumerando las 15 palabras no nulas

m (bits)	V = m·g	Peso
1000	1101000 1101000	6
0100	0110100 0110100	6
1100	1011100 1011100	8
0010	0011010 0011010	6
1010	1110010 1110010	8
0110	0101110 0101110	8
1110	1000110 1000110	6
0001	0001101 0001101	6

m (bits)	V = m·g	Peso
1001	1100101 1100101	8
0101	0111001 0111001	8
1101	1010001 1010001	6
0011	0010111 0010111	8
1011	1111111 1111111	14
0111	0100011 0100011	6
1111	1001011 1001011	8

Pesos: 6 (×7), 8 (×7), 14 (×1) → d_{min} = 6, t = 2, detecta 5 (corrige 2 y detecta 3 a la vez) · redundancia n/k = 14/4 = 3,5

⚠ La tabla de pesos de la resolución 2025 de alumna no coincide con la real (por ejemplo da 7 u 8 para palabras que son desplazamientos de g —m = 1, x, x^2— y por lo tanto pesan 6, y da 9 para la 1011, que en realidad es todo unos, peso 14). La conclusión d_{min} = 6 sí es correcta. Ver sección 8.

Paso 3 · G y H

Forma 1 – por desplazamientos de g (la de la resolución 2025; es la más rápida):

G (4 × 14): filas g(x), x·g, x^2·g, x^3·g

```
1 1 0 1 0 0 0 1 1 0 1 0 0 0
0 1 1 0 1 0 0 0 1 1 0 1 0 0
0 0 1 1 0 1 0 0 0 1 1 0 1 0
0 0 0 1 1 0 1 0 0 0 1 1 0 1
```

Para H hay que usar el **recíproco** de h(x): h = `11101` invertido es `10111`; las filas de H son sus desplazamientos (10 filas = n−k):

H (10 × 14) con filas = desplazamientos de 10111

```
1 0 1 1 1 0 0 0 0 0 0 0 0 0
0 1 0 1 1 1 0 0 0 0 0 0 0 0
0 0 1 0 1 1 1 0 0 0 0 0 0 0
0 0 0 1 0 1 1 1 0 0 0 0 0 0
0 0 0 0 1 0 1 1 1 0 0 0 0 0
0 0 0 0 0 1 0 1 1 1 0 0 0 0
0 0 0 0 0 0 1 0 1 1 1 0 0 0
0 0 0 0 0 0 0 1 0 1 1 1 0 0
0 0 0 0 0 0 0 0 1 0 1 1 1 0
0 0 0 0 0 0 0 0 0 1 0 1 1 1
```

(Comprobado: $G \cdot H^T = 0$. Usar h sin invertir **no** da una H ortogonal a G: ver sección 8.)

Forma 2 – sistemática G = [P | I_4], H = [I_{10} | P^T] (la que permite leer los chequeadores):

G (4 × 14)

```
1 1 0 1 0 0 0 1 1 0 | 1 0 0 0
0 1 1 0 1 0 0 0 1 1 | 0 1 0 0
1 1 1 0 0 1 0 1 1 1 | 0 0 1 0
1 0 1 0 0 0 1 1 0 1 | 0 0 0 1
```

H (10 × 14)

```
1 0 0 0 0 0 0 0 0 0 | 1 0 1 1
0 1 0 0 0 0 0 0 0 0 | 1 1 1 0
0 0 1 0 0 0 0 0 0 0 | 0 1 1 1
0 0 0 1 0 0 0 0 0 0 | 1 0 0 0
0 0 0 0 1 0 0 0 0 0 | 0 1 0 0
0 0 0 0 0 1 0 0 0 0 | 0 0 1 0
0 0 0 0 0 0 1 0 0 0 | 0 0 0 1
0 0 0 0 0 0 0 1 0 0 | 1 0 1 1
0 0 0 0 0 0 0 0 1 0 | 1 1 1 0
0 0 0 0 0 0 0 0 0 1 | 0 1 1 1
```

Chequeadores como combinación de bits de información ($i_{0..3}$ = posiciones 10..13):

c_0	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9
$i_0 \oplus i_2 \oplus i_3$	$i_0 \oplus i_1 \oplus i_2$	$i_1 \oplus i_2 \oplus i_3$	i_0	i_1	i_2	i_3	$i_0 \oplus i_2 \oplus i_3$	$i_0 \oplus i_1 \oplus i_2$	$i_1 \oplus i_2 \oplus i_3$

Paso 4 · Decodificar (2025): R(x) = 11110001101000

R(x) = $x^{10} + x^8 + x^7 + x^3 + x^2 + x + 1$. Se divide por g(x):

Paso	Resto parcial (mayor potencia a la izquierda)	Término del cociente	Divisor desplazado	Resto nuevo (suma mod 2)
1	`10110001111`	1	`10110001011`	`00000000100`

Resto = $\mathbf{x^2} \neq 0$ → no es válida. Como x^2 tiene grado menor que n−k = 10, el síndrome de un error simple en la posición i < 10 es x^i mismo → **error en el bit 2**. Se invierte: `11110001101000` → `11010001101000` = g(x), múltiplo de g → válida (resto 0) ✓.

Corregida: 11010001101000 (información 0 0 0 1 → corresponde a m(x) = 1)

Paso 5 · R(x) = $x^8 + x^4$ (2012 T2 / 2013 T1)

R(x) = $x^8 + x^4$ es no nulo y de grado 8 < 10 → **no es múltiplo de g** → no válida (atajo de 1.5). La división no hace falta: R ya es menor que g, así que el **resto es R(x) mismo** = $x^8 + x^4$.

Como $d_{min} = 6 \rightarrow t = 2$, se busca un patrón de peso ≤ 2 con ese resto: para posiciones < 10 el resto de x^i es x^i, de modo que $e = x^8 + x^4$ (errores en las posiciones 4 y 8). Corregida: R + e = **0** (la palabra nula, que es válida). Es la única corrección posible con t = 2.

R no válida → errores en x^4 y x^8 → palabra corregida = 00000000000000 (información 0000)

Paso 6 · Entrelazado

t = 2: con L = 20 → **40 bits** de ráfaga (2012 T2, 2013 T1); con L = 25 → **50 bits** (2025).

4.4 Código (15, 4) con h(x) = $x^4 + x + 1$ 2012 T1

Enunciado: (15, K?, d_{min}?), h(X) = X^4+X+1. a) g(X). b) capacidades. c) chequeadores. d) ¿R(x) = $X^8+X^4+X^2$ es válido? e) ráfaga con 20 códigos.

k = 4, n−k = 11. g(x) = $(x^{15}+1)/h(x)$:

Paso	Resto parcial (mayor potencia a la izquierda)	Término del cociente	Divisor desplazado	Resto nuevo (suma mod 2)
1	`1000000000000001`	x^{11}	`1001100000000000`	`0001100000000001`
2	`0001100000000001`	x^8	`0001001100000000`	`0000101100000001`
3	`0000101100000001`	x^7	`0000100110000000`	`0000001010000001`
4	`0000001010000001`	x^5	`0000001001100000`	`0000000011100001`
5	`0000000011100001`	x^3	`0000000010011000`	`0000000001111001`
6	`0000000001111001`	x^2	`0000000001001100`	`0000000000110101`
7	`0000000000110101`	x	`0000000000100110`	`0000000000010011`
8	`0000000000010011`	1	`0000000000010011`	`0000000000000000`

g(x) = $x^{11} + x^8 + x^7 + x^5 + x^3 + x^2 + x + 1$ → `111101011001` (8 términos)

d_{min}: las 15 palabras no nulas pesan **todas 8** (es el código «símplex»): **d_{min} = 8 → t = 3**, detecta 7 (corrige 3 y detecta 4 a la vez), redundancia 15/4 = 3,75.

Chequeadores (P = x^{11+i} mod g; información $i_{0..3}$ en las posiciones 11..14):

c_0	c_1	c_2	c_3	c_4	c_5
$i_0 \oplus i_3$	$i_0 \oplus i_1 \oplus i_3$	$i_0 \oplus i_1 \oplus i_2 \oplus i_3$	$i_0 \oplus i_1 \oplus i_2$	$i_1 \oplus i_2 \oplus i_3$	$i_0 \oplus i_2$

c_6	c_7	c_8	c_9	c_{10}
$i_1 \oplus i_3$	$i_0 \oplus i_2 \oplus i_3$	$i_0 \oplus i_1$	$i_1 \oplus i_2$	$i_2 \oplus i_3$

G = [P | I_4] y H = [I_{11} | P^T] con P cuyas filas son x^{11}, x^{12}, x^{13}, x^{14} mod g:

G (4 × 15)

```
1 1 1 1 0 1 0 1 1 0 0 | 1 0 0 0
0 1 1 1 1 0 1 0 1 1 0 | 0 1 0 0
0 0 1 1 1 1 0 1 0 1 1 | 0 0 1 0
1 1 1 0 1 0 1 1 0 0 1 | 0 0 0 1
```

¿Es válido R(x) = $x^8 + x^4 + x^2$? Es no nulo y de grado 8 < 11 → **no es múltiplo de g** → no válido; el resto es R mismo. Se busca un patrón de peso ≤ t = 3 con ese síndrome: e = $x^2 + x^4 + x^8$ (posiciones 2, 4, 8; peso 3 ≤ 3). Corregida: R + e = **0**. Es única, porque con d_{min} = 8 dos patrones de peso ≤ 3 distintos no pueden dar el mismo síndrome (su diferencia sería una palabra de peso ≤ 6 < 8).

Corregida = palabra nula (3 errores corregidos) · Ráfaga con L = 20: 3·20 = **60 bits**

5. Entrelazado y ráfagas (Etapa 7)

Los códigos de bloque están pensados para errores **aleatorios** (independientes). Una **ráfaga** (errores en bits consecutivos por una causa común: un rayón en un CD, un desvanecimiento) puede superar la capacidad t de una sola palabra.

Entrelazar L palabras: se escriben las L palabras como filas de una matriz y se transmite **por columnas** (primer bit de la palabra 1, primer bit de la palabra 2, …, primer bit de la palabra L; después los segundos bits…). Si el canal daña una ráfaga de B bits consecutivos, cada palabra recibe como máximo ⌈B/L⌉ errores. Se puede corregir si ⌈B/L⌉ ≤ t, es decir:

Ráfaga corregible = t · L bits (y detectable: $(d_{min}-1)\cdot L$)

Ejemplo con L = 3, t = 1 (se transmiten las columnas)

```
palabra A:  a1 a2 a3 a4 a5 a6 a7
palabra B:  b1 b2 b3 b4 b5 b6 b7
palabra C:  c1 c2 c3 c4 c5 c6 c7

se envía :  a1 b1 c1 | a2 b2 c2 | a3 b3 c3 | …
ráfaga de 3 bits seguidos (a2 b2 c2) → 1 error por palabra → t = 1 los corrige
```

Código	Palabras entrelazadas L	Ráfaga corregible t·L (bits)	Parcial
(12, 3), t = 2	7	14	2010 (teórica)
(12, 3), t = 2	20	40	2013 T2
(14, 4), t = 2	20	40	2012 T2, 2013 T1
(14, 4), t = 2	25	50	2025
(15, 4), t = 3	20	60	2012 T1

«¿Qué nuevas capacidades adquiere el código?»

Sigue corrigiendo t errores aleatorios por palabra, pero **adquiere capacidad de corrección de ráfagas de hasta t·L bits**. La tasa k/n no cambia (no se agregan chequeadores); el costo es **latencia y memoria** (hay que juntar las L palabras antes de transmitir/decodificar) y que una ráfaga mayor a t·L ya no se corrige garantizadamente.

6. Reed–Solomon: la tabla (Etapa 8)

Reed–Solomon trabaja con **símbolos (bytes)** de m bits sobre un campo de Galois GF(2^m). Por eso es muy potente contra ráfagas: da lo mismo que un byte tenga 1 o m bits dañados, cuenta como **un** error de símbolo.

Reglas para completar la tabla

- **Tamaño del campo** q = 2^m; **bits por byte** = m; **grado del polinomio que genera el campo** = m.
- **Longitud de palabra** n = q − 1 (bytes).
- **Bytes chequeadores** n − k = 2t; **bytes de información** k = n − 2t; **t** = capacidad de corrección en bytes.
- **Ráfaga con L códigos entrelazados** = t·L bytes.

2012 T2 **Tabla**

Tamaño del campo GF(q)	Grado del pol. del campo	n (bytes)	bits / byte	bytes chequeadores	bytes información	t	7 códigos entrelazados
2048	11	2047	11	200	1847	100 bytes	700 bytes
512	9	511	9	50	461	25 bytes	175 bytes

Tamaño del campo GF(q)	Grado del pol. del campo	n (bytes)	bits / byte	bytes chequeadores	bytes información	t	7 códigos entrelazados
128	7	127	7	10	117	5 bytes	35 bytes

Fila 1 (dato: q = 2048, t = 100): $2048 = 2^{11}$ → m = 11, n = 2047, 2t = 200, k = 2047 − 200 = 1847, ráfaga 7·100 = 700 bytes.
Fila 2 (dato: grado 9, k = 461): $q = 2^9 = 512$, n = 511, 2t = 511 − 461 = 50 → t = 25, ráfaga 7·25 = 175 bytes.
Fila 3 (dato: n = 127, 10 chequeadores): $q = 128 = 2^7$, m = 7, 2t = 10 → t = 5, k = 117, ráfaga 7·5 = 35 bytes.

Datos del enunciado: fila 1 = tamaño del campo 2048 y capacidad 100 bytes; fila 2 = grado del polinomio del campo 9 y 461 bytes de información; fila 3 = longitud de palabra 127 y 10 bytes chequeadores. Cada fila queda determinada por esos dos datos.

Tabla de «Preguntas teóricas» (verificada)

Capacidad	Grado g(x) = 2t	Grado campo	Tamaño	Longitud V(x)	Bits/byte	Bytes de check	Bytes de info
Corrector doble	4	5	32	31	5	4	27
Corrector triple	6	5	32	31	5	6	25
Corrector triple	6	4	16	15	4	6	9
Corrector de 7 bytes	14	8	256	255	8	14	241

Regla: g(x) tiene grado 2t (= bytes de chequeo). Ej.: corrector de 7 bytes → 2t = 14; GF(256) → n = 255 → k = 241 ✓.

7. Teóricas de control de error con respuesta

1. Si se entrelazan n códigos, ¿qué nuevas capacidades adquiere el código?
Corrige ráfagas de longitud t·L (t = capacidad de corrección por palabra, L = cantidad de palabras entrelazadas); también multiplica por L la longitud de ráfaga detectable. No cambia t ni la tasa k/n. Ver sección 5.

2. Propiedades del polinomio generador de un código cíclico.
(1) es de grado n−k; (2) es factor de x^n+1; (3) coeficiente 1 en el término independiente y en el de mayor grado; (4) es el de menor grado y primo; (5) toda palabra válida es múltiplo de g(x) y todo múltiplo de g(x) (grado menor que n) es palabra válida; (6) toda rotación de una palabra válida es otra palabra válida.

3. ¿Qué relación tienen G y H? ¿Qué significa?
Son **ortogonales**: $G \cdot H^T = 0$. Si $G = [I \mid P]$ entonces $H = [P^T \mid I]$; si $G = [P \mid I]$ entonces $H = [I \mid P^T]$. Toda palabra válida pertenece al subespacio generado por las filas de G, y H genera el subespacio perpendicular: el producto escalar de una palabra válida por cada fila de H da 0; si hubo error, el síndrome $S = R \cdot H^T$ es ≠ 0 y coincide con la suma de las columnas de H en las posiciones erróneas.

4. ¿Un código cíclico de capacidad 10 ($d_{min} = 10$), qué puede hacer?
Como corrector: $d_{min} \geq 2t+1 \rightarrow t = 4$, corrige hasta 4 (y detecta 5 a la vez). Como detector: detecta $d_{min}-1 = 9$.

5. Capacidad de un código ISBN.
$d_{min} = 2$: detecta un error en una sola posición (y el intercambio de dos dígitos adyacentes en ISBN-10); no corrige. Dos errores que se compensan producen otro ISBN válido: error indetectable.

6. ¿Cómo se calcula el dígito faltante de un ISBN?
Pesos 10…1, suma de dígito × peso ≡ 0 (mod 11); resolver $w \cdot x \equiv -S \pmod{11}$. Ver sección 2.

7. Necesidad de la base prima.
Con base prima todo peso no nulo tiene inverso, así que la ecuación de decodificación $w \cdot x \equiv c$ tiene **solución única**: no hay ambigüedad al decodificar.

8. ¿Por qué $d_{min} \geq 2t+1$?
Para corregir t errores, las «esferas» de radio t alrededor de cada palabra válida no deben superponerse: dos palabras a distancia ≥ 2t+1 dejan cada palabra recibida a lo sumo a distancia t de una sola. Como detector basta $d_{min}-1$ porque ninguna combinación de ≤ $d_{min}-1$ errores convierte una palabra válida en otra.

9. ¿Qué sucede en la decodificación con más errores de los que corrige el código?
Si el patrón tiene peso ≤ $d_{min}-1$ el síndrome es ≠ 0 y se detecta (no se corrige); con peso mayor el patrón puede coincidir con otra palabra válida y no detectarse, o ser «corregido» al lado equivocado.

10. ¿Qué distancia mínima tiene un arreglo rectangular?
$d_{min} = d_1 \cdot d_2$ (producto de las distancias del código por filas y del código por columnas). Ver 3.2.

8. ⚠ Errores encontrados en resoluciones de alumnos (y cómo evitarlos)

Estas resoluciones son útiles de referencia, pero al recalcular todo aparecieron los siguientes puntos. Conviene no copiarlos.

Dónde	Qué dice la resolución	Qué da el cálculo
2025 – 1a (B. Melani)	Tabla de pesos con valores 6, 7, 8 y 9 para las 15 palabras.	Los desplazamientos de g ($x \cdot g$, $x^2 \cdot g$, …) pesan siempre 6. Los pesos reales son 6 (×7), 8 (×7) y 14 (×1). La conclusión $d_{min} = 6$ es correcta.
2025 – 1b (B. Melani)	H con filas = desplazamientos de h = `11101`.	**No es ortogonal a G**: 23 de los 40 productos fila de G × fila de H dan 1 (p. ej. G fila 1 · H fila 3 = 1). Hay que usar el **recíproco** `10111` (o la forma sistemática $[I \mid P^T]$); con eso $G \cdot H^T = 0$.
2025 – 1c, 1d, 1e	Resto $x^2 \rightarrow$ error en el bit 2; corregida = g(x); 25 códigos → 50 bits.	✓ Correcto (verificado).
2025 – 2 (ISBN-13)	Suma 103 → inválido; con verificador = 0 queda válido.	✓ Correcto. Agregar que solo se corrige bajo la hipótesis de que el erróneo es el verificador (13 posibles correcciones si no).

Dónde	Qué dice la resolución	Qué da el cálculo
2025 – 3 (arreglo)	$d_{min} = d_1 \cdot d_2 = 3 \cdot 2 = 6$, $t = 2$. Palabra inválida por fila 2 y columna 2.	La fórmula es correcta; con BP de fila y $d_1 = 4$ se obtiene 8 (7 sin esquina), $t = 3$. Además falla también la columna 3. Confirmar con la cátedra qué esperan.
2020 – 2A (J. Arce)	ISBN 3-540-?3218-9 → ? = 6.	Error de suma: S = 158 (no 113); $6x \equiv 7 \pmod{11}$ → **? = 3**. Con 3 el total es 176 = 16·11 (el mismo que muestra la diapositiva 9).
2020 – 1 (J. Arce)	$g(x) = x^9+x^8+x^5+x^4+x+1$, G de 3×12, $d_{min} = 6$, corrige 2 y detecta 5, $H = [P^T \mid I]$.	✓ g(x), G y capacidad correctos. Su H, tal como se lee en la foto, también resulta ortogonal a su G en este código en particular; la pareja general es $G = [P \mid I] \leftrightarrow H = [I \mid P^T]$ (o $G = [I \mid P] \leftrightarrow H = [P^T \mid I]$), así que conviene no mezclar los órdenes.

Errores típicos a evitar

- Dividir con el string en el orden del enunciado (mayor potencia a la derecha) y alinear mal. Invertí antes de dividir.
- Usar h(x) sin invertir para armar H cuando G viene de desplazar g(x). El recíproco de h es el que sirve.
- Calcular d_{min} con una tabla a medias. Si enumerás, hacelo con todas las palabras y chequeá que los desplazamientos de g pesan lo mismo que g.
- Decir que «se corrige» un ISBN sin aclarar la hipótesis. Un ISBN se detecta; se corrige solo si se sabe qué dígito es el erróneo.
- Olvidar que un polinomio no nulo de grado menor que $n-k$ jamás es palabra válida.

9. Ejercicios propuestos (con soluciones al final)

Resolvelos sin mirar la sección 9.2. Están pensados como variantes de lo que ya viste.

9.1 Enunciados

0. **P0 – capacidad.** Un código cíclico tiene $d_{min} = 9$. ¿Qué corrige, qué detecta y qué ráfaga corrige si se entrelazan 30 palabras? Repetí para $d_{min} = 7$ y 12.
1. **P1 – ISBN-10.** Hallá el dígito faltante: (a) 0-306-4?615-2 (b) 0-13-11?362-8 (c) 0-262-0?384-4.
2. **P2 – ISBN-13.** Indicá si 978-0-13-110363-7 es válido. Si no, corregilo suponiendo que el erróneo es el verificador. (El libro real es 978-0-13-110362-7: ¿qué observás?)
3. **P3 – Hamming (7,4).** Recibido `0 1 1 1 0 0 0` ($C_1\ C_2\ I_1\ C_3\ I_2\ I_3\ I_4$). Calculá el síndrome, corregí y dá la información.
4. **P4 – cíclico (7,4), $g = x^3+x+1$.** (a) Codificá sistemáticamente la información `1011`. (b) Decodificá `0001111` y `0110101` (mayores potencias a la derecha), corrigiendo.
5. **P5 – variante de 2012 T1.** $n = 15$, $h(x) = x^4 + x^3 + 1$. Hallá k, g(x), d_{min}, capacidad y la ráfaga con 10 códigos entrelazados.
6. **P6 – Reed–Solomon.** Completá: (a) t = 20 bytes, 10 bits por byte, 75 códigos entrelazados; (b) t = 40 bytes, n = 255, 75 códigos; (c) n = 63, 24 bytes chequeadores.
7. **P7 – teóricas.** Escribí de memoria: propiedades de g(x), relación G–H, y por qué la base prima.
8. **P8 – simulacro.** Resolvé en 60 minutos y sin apuntes: control de error de 2020 (sección 4.2 con todo el inciso E) + ISBN-13 y arreglo de 2025.

9.2 Soluciones

P0. $d_{min} = 9 \rightarrow$ corrige t = 4; como detector puro detecta 8; ráfaga 4·30 = **120 bits**. $d_{min} = 7 \rightarrow$ corrige 3, detector puro 6, ráfaga 3·30 = 90. $d_{min} = 12 \rightarrow$ corrige 5, detector puro 11 (o corrige 5 y detecta 6 a la vez), ráfaga 5·30 = 150.

P1. (a) S = 132, peso 5 → **0** (b) S = 88, peso 5 → **0** (c) S = 128, peso 5 → **3**. (Son tres ISBN reales.)

P2. Suma total = 83 → 83 mod 10 = 3 ≠ 0 → **no válido**. Suma de los 12 primeros = 76 → verificador = 4 → «corrige» a 978-0-13-110363-4. Pero el libro real es …362-7: el error estaba en el dígito 12 (3 en vez de 2), no en el verificador. **La corrección bajo hipótesis puede ser incorrecta**: un ISBN se detecta, no se corrige a ciegas.

P3. $S_1 = 1$, $S_2 = 0$, $S_3 = 1 \rightarrow (S_3S_2S_1) = 101$ = posición **5** → corregida `0111100`; información $I_1I_2I_3I_4$ = `1100`.

P4. (a) $i(x) = 1 + x^2 + x^3$; $x^3 \cdot i(x) = x^6 + x^5 + x^3$; resto (÷g) = 1 → V = `1001011` (chequeadores `100`, información `1011`). (b) `0001111`: resto $x^2 + x + 1 = x^5 \bmod g \rightarrow$ error en el bit 5 → `0001101`, información `1101`. `0110101`: resto $x^2 + 1 = x^6 \bmod g \rightarrow$ error en el bit 6 → `0110100`, información `0100`.

P5. k = 4, n−k = 11. $g(x) = x^{11} + x^{10} + x^9 + x^8 + x^6 + x^4 + x^3 + 1$ (bits `100110101111`, 8 términos). Las 15 palabras no nulas pesan 8 → $\mathbf{d_{min} = 8}$**, t = 3**, detecta 7. Ráfaga con 10 códigos: 3·10 = **30 bits**.

P6. (a) m = 10 → q = 1024, n = 1023, 2t = 40, k = 983, ráfaga 20·75 = **1500 bytes**. (b) n = 255 → q = 256, m = 8, 2t = 80, k = 175, ráfaga 40·75 = **3000 bytes**. (c) n = 63 → q = 64, m = 6, 2t = 24 → t = 12, k = 39.

P7. Ver secciones 1.5, 7 (preguntas 2, 3 y 7) y 2.3. Autoevaluate: ¿nombraste las 6 propiedades? ¿mencionaste $G \cdot H^T = 0$ y las dos parejas [I | P] / [P^T | I]? ¿pusiste el contraejemplo base 10?

P8. Criterios: (1) k = 3 y g(x) por división; (2) $d_{min} = 6$ por enumeración, t = 2, detecta 5; (3) G, H verificando $G \cdot H^T = 0$; (4) decodificar 110011001110 con corrección; (5) error triple se detecta y no se corrige; (6) ISBN-13: suma 103, no válido, verificador 0; (7) arreglo: $d_{min} = d_1 \cdot d_2$ y localización fila–columna.

10. Chuleta de una página

Capacidad

$t = \lfloor (d_{min}-1)/2 \rfloor$ · detecta $d_{min}-1$ · mixto: $d_{min} \geq t+s+1$
Lineal: d_{min} = peso mínimo de palabra no nula.
Cíclico: $d_{min} \leq$ #términos de g.

Cíclicos

k = grado h · n−k = grado g · $g = (x^n+1)/h$
V = i·g (no sist.) · $V = x^{n-k}\cdot i + r$ (sist.), $r = x^{n-k}\cdot i \bmod g$
Decodificar: r = R mod g; r = 0 válida; si no, e (peso ≤ t) con e mod g = r; V = R+e.
Grado(R) < n−k y R ≠ 0 ⇒ no válida (resto = R).

Matrices

G = [P | I] con fila i de $P = x^{n-k+i} \bmod g$
$H = [I \mid P^T] \cdot G \cdot H^T = 0$
Si G = filas g, xg, x^2g..., H = desplazamientos del **recíproco** de h.
Chequeador j = suma de los i_m con P[m][j] = 1.
Síndrome = suma de columnas de H en las posiciones erróneas.

Entrelazado

Ráfaga corregible = t·L · detectable = $(d_{min}-1)\cdot L$

Hamming (7,4)

$C_1\ C_2\ I_1\ C_3\ I_2\ I_3\ I_4$ · S_1: pos 1,3,5,7 · S_2: 2,3,6,7 · S_3: 4,5,6,7
$(S_3S_2S_1)_2$ = posición del error · $d_{min} = 3$
+ BP: $d_{min} = 4$ · S≠0 y BP falla → corrige · S≠0 y BP ok → doble.

Arreglos

$d_{min} = d_1 \cdot d_2$ · falla fila + falla columna → error en la intersección.

ISBN

ISBN-10: pesos 10…1, Σ ≡ 0 (mod 11); w·x ≡ −S. Inversos mod 11: 1→1, 2→6, 3→4, 4→3, 5→9, 6→2, 7→8, 8→7, 9→5, 10→10.
ISBN-13: pesos 1,3,1,3…, Σ ≡ 0 (mod 10); verificador = $(10 - S_{12} \bmod 10) \bmod 10$.
$d_{min} = 2$: detecta 1 error, no corrige. Base prima ⇒ solución única.
Base 37 (alfanumérico): suma de valores mod 37, sin pesos.

Reed–Solomon

$q = 2^m$; n = q−1; bits/byte = m; grado del campo = m; n−k = 2t; ráfaga = t·L bytes.

Los tres códigos de siempre

(n,k)	h(x)	g(x) (bits)	d_{min}	t	L = 20
(12,3)	x^3+x^2+x+1	`1100110011`	6	2	40
(14,4)	x^4+x^2+x+1	`11010001101`	6	2	40
(15,4)	x^4+x+1	`111101011001`	8	3	60